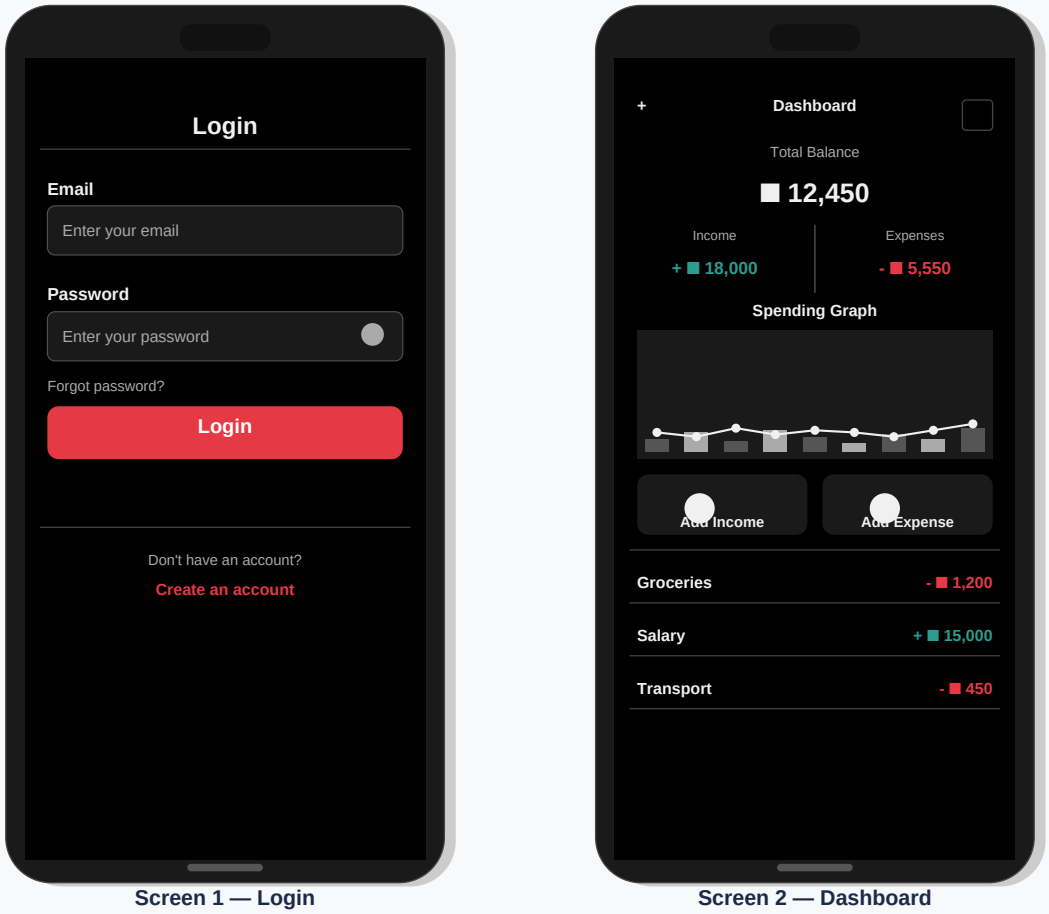
COMPREHENSIVE TECHNICAL PRODUCT REQUIREMENTS DOCUMENT

Document Version	2.0 — Full Release
Prepared For	Engineering, Design & QA Teams
Platform	React Native (Expo) — iOS & Android
Backend	Supabase (PostgreSQL + Auth + Edge Functions)
Currency	Indian Rupee (■) — i18n ready
Confidentiality	Internal Use Only
Date	2026

This document serves as the single source of truth for all development decisions. All engineers, designers, and QA team members should refer to this PRD before implementation of any feature.

UI. APP SCREEN DESIGN REFERENCE

The following mockups illustrate the target visual design for the Expense Tracker application. These screens reflect the agreed design language: OLED-optimised black background (#000000), Crimson Red (#E63946) for expenses and CTAs, and Emerald Green (#2A9D8F) for income. Developers must implement all screens to match these specifications.



Screen 1 — Login

Screen 2 — Dashboard

Figure 1: App Design Reference — Login Screen (left) and Dashboard Screen (right). Reproduced from the approved design mockup provided by the product team.

Key Design Specifications from Reference Screens

Element	Login Screen	Dashboard Screen
Background	#000000 — full bleed black	#000000 — full bleed black
Header	No top bar; 'Login' title centered, white bold	'Dashboard' centered; + icon left; bell icon right
Input Fields	Dark card bg (#1A1A1A), gray border (#333), gray placeholder text	N/A
Primary Button	Full-width red (#E63946), 12dp radius, 'Login'	N/A
Balance Display	N/A	Large white bold (■ 12,450); 'Total Balance' in light gray above
KPI Row	N/A	Income (green +■18,000) | Expenses (red -■5,550) split by vertical

Element	Login Screen	Dashboard Screen
Spending Graph	N/A	Bar chart (gray bars) + line chart (white) combo; 'Spending Graph' l
Action Buttons	N/A	2 equal dark-card buttons: 'Add Income' (+) and 'Add Expense' (-)
Transaction List	N/A	Category name bold white left; amount color-coded right; light divid
Footer Link	'Don't have an account?' gray + 'Create an account' red bold	N/A

1. PROJECT VISION & UX LANGUAGE

The Expense Tracker is a mobile-first personal finance application built for speed, clarity, and offline resilience. It empowers users to record, categorise, and analyse every rupee of income and expenditure with minimal friction. The product vision rests on three pillars: **Speed** (sub-100 ms interactions), **Clarity** (high-contrast, OLED-optimised dark UI), and **Trust** (end-to-end encryption, biometric security, GDPR compliance).

1.1 Color Palette (App UI — not this document)

Token	Hex Code	Usage	Screen Reference
Deep Black	#000000	App background (OLED battery saving)	All screens
Card Dark	#111111	Card surfaces, modals	Dashboard cards
Card Dark 2	#1A1A1A	Inputs, alternating rows	Login inputs
Crimson Red	#E63946	Expenses, CTA buttons, accent	Login button, expense amounts
Emerald Green	#2A9D8F	Income, success states, positive KPIs	Income amounts
Light Gray	#AAAAAA	Secondary labels, placeholders	Input placeholder text
Medium Gray	#555555	Dividers, subtle borders	Row separators
Off-White	#F0F0F0	Primary body text on dark bg	All screen text

1.2 Typography Scale

Font family: **Inter** (variable, via Expo Google Fonts). Four-step modular scale: Display (32 sp / Bold) for total balance hero; Title (20 sp / SemiBold) for screen headers; Body (15 sp / Regular) for transaction rows; Caption (12 sp / Regular) for timestamps. All sizes respect OS accessibility 'Large Text' via React Native's *allowFontScaling* prop.

1.3 Screen Inventory

#	Screen Name	Key Components
1	Splash / Onboarding	Animated logo, 3-step value-prop carousel, Get Started CTA
2	Login	Email + Password fields, eye-toggle, Forgot Password, red Login button, Create Account link (see desig
3	Register	Name, Email, Password, Confirm Password, Terms checkbox, Register CTA
4	Dashboard	Total Balance hero, Income/Expense KPI row, Spending Graph (bar+line), Add Income / Add Expense l
5	Add Transaction	Type toggle (Income/Expense), Amount input, Category picker, Date picker, Note field, Save
6	Transaction Detail	Full transaction info, Edit / Delete actions, category icon, timestamp
7	Analytics	Monthly/Weekly/Daily toggle, Category pie chart, Trend line chart, Top-5 expense breakdown
8	Group Splits	Create Group, Add Members, Split Mode selector, Settlement summary
9	Settings	Profile, Notifications, Biometrics toggle, Export Data, Delete Account
10	Notifications	Bell icon badge, budget warnings, settlement reminders, sync status

2. ONLINE-TO-OFFLINE LIFECYCLE

The application operates as an **Offline-First** system. Every user action writes to the local SQLite database first. A background sync engine asynchronously replicates changes to Supabase when connectivity is detected, ensuring zero data loss and instant UI responsiveness in all network conditions.

2.1 Authentication Flow

Step	Action	Technical Detail
1	Submit credentials	Email + Password POST to Supabase Auth /auth/v1/token
2	Receive tokens	Access Token (JWT, 1h expiry) + Refresh Token (30-day sliding window)
3	Secure local storage	Both tokens written to Expo SecureStore (iOS Keychain / Android Keystore)
4	Bootstrap local DB	On first login, pull user profile and last 90 days of transactions into SQLite
5	Silent token refresh	React Query re-fetches token 5 min before expiry using stored refresh token
6	Offline fallback	If refresh fails due to no connectivity, allow app entry from cached token until expiry
7	Biometric gate (opt-in)	After token validation, FaceID/TouchID prompt via Expo LocalAuthentication
8	Logout	Clear SecureStore, purge React Query cache, soft-delete local SQLite session row

2.2 Write Lifecycle — Offline-First CRUD

- **CREATE:** New transaction inserted into SQLite with status='pending' and a client-generated UUID.
- **READ:** UI always reads from SQLite. No network blocking. Supabase used only for initial hydration.
- **UPDATE:** SQLite row updated in place; updated_at refreshed; status set to 'pending'.
- **DELETE:** Soft-delete (is_deleted=1) in SQLite; hard-delete propagated to Supabase on next sync.
- **CONFLICT:** Last-write-wins based on updated_at timestamp. Server record wins on tie.

2.3 Background Sync Engine — 8-Phase Cycle

Phase	Description
1. Connectivity check	NetInfo.fetch() — abort and queue retry with exponential back-off (1s, 2s, 4s, max 60s) if offline
2. Token validation	Verify JWT expiry; silently refresh if needed before any API calls
3. Push pending rows	SELECT * FROM transactions WHERE status='pending' ORDER BY created_at ASC LIMIT 100
4. Upsert to Supabase	Batch upsert via /rest/v1/transactions with conflict-resolution on id column
5. Mark synced	UPDATE transactions SET status='synced' WHERE id IN (successfully_upserted_ids)
6. Pull remote changes	Fetch rows WHERE updated_at > last_sync_ts from Supabase; merge into SQLite
7. Update sync stamp	Write current UTC timestamp to users.sync_ts in SQLite
8. Notify UI	Emit Zustand store event; React Query invalidates relevant query keys; UI refreshes

3. ADVANCED GROUP SPLITTING

The Group Splitting module supports expense-sharing among 2 or more participants with a **Debt Minimisation Algorithm** that calculates the minimum number of transactions needed to settle all debts within a group — at most N-1 transactions for N participants.

3.1 Split Modes

Mode	Description	Validation Rule	UI Control
Equal	Total divided equally by N members	Automatic — no input needed	Toggle switch
Percentage	Each member assigned a % share	Sum of all % must equal 100%	% input per member
Shares	Weighted integer shares per member	Minimum 1 share per member	Stepper per member
Custom	Manual fixed amount per member	Sum must equal transaction total	Currency input per member

3.2 Debt Minimisation Algorithm

- Compute net balance for each participant: net[i] = total_paid[i] - total_owed[i].
- Separate into creditors (net > 0) and debtors (net < 0).
- Greedily match largest debtor to largest creditor until all balances reach zero.
- Each match generates exactly one settlement transaction.
- Result: at most N-1 settlement transactions for any group of N participants.

3.3 Group Lifecycle States

State	Trigger	Permissions
Active	Group created	All members: add expenses, view settlements
Settling	Admin taps 'Settle Up'	Members mark payments; admin can override
Settled	All balances = 0	Read-only archive; export to PDF available
Archived	Admin archives group	No further edits; retrievable from history

3.4 Group Splitting UI Flow

Step	Screen Element	Action
1	Dashboard → Group Splits	User taps '+ New Group'
2	Name & Members	Enter group name; search users by email; send invite
3	Add Group Expense	Enter amount, description, category, date
4	Choose Split Mode	Toggle Equal / Percentage / Shares / Custom
5	Review Split	Preview each member's share; confirm or edit
6	Settlement View	Tap 'Settle Up' to see minimum transactions; mark as paid

4. DATABASE ARCHITECTURE (SQLITE + SUPABASE)

All data is stored locally in **Expo SQLite** (WAL mode, AES-256 encrypted). The cloud replica lives in **Supabase (PostgreSQL 15)** with Row Level Security (RLS) policies ensuring users only access their own data.

4.1 Table Definitions

Table	Primary Key	Key Columns	Notes
users	id (UUID)	email, display_name, avatar_url, local_settings JSONB, sync_ts	1 row per registered user
categories	id (INT)	name, icon_key, color_hex, type (income/expense), is_default, user_id	Seeded defaults + user-created custo
transactions	id (UUID)	user_id, type, cat_id, amount DECIMAL(12,2), note, date, status, is_deleted, created_at, updated_	One table, soft-delete pattern
budgets	id (UUID)	user_id, cat_id, amount_limit, period, start_date, alert_pct	Budget alert at alert_pct threshold
groups	id (UUID)	name, created_by, created_at, status (active/settled/archived)	Group expense containers
group_members	group_id+user_id	role (admin/member), joined_at, invite_status	Composite primary key
group_expenses	id (UUID)	group_id, paid_by, amount, description, cat_id, date, split_method	Links to splits table
splits	id (UUID)	expense_id, user_id, amount_owed, amount_paid, is_settled	One row per member per expense
notifications	id (UUID)	user_id, type, title, body, is_read, created_at, action_url	In-app notification log

4.2 Row Level Security Policies (Supabase)

- **transactions:** USING (auth.uid() = user_id) — users read/write only their own rows.
- **groups:** USING (id IN (SELECT group_id FROM group_members WHERE user_id = auth.uid())) — members only.
- **splits:** Joins through group_members to verify group membership before any access.
- **budgets:** USING (auth.uid() = user_id).
- Service-role key used only in Edge Functions; never exposed to the client app.

4.3 Database Indexes

Index Name	Table	Columns	Type
idx_trans_user_date	transactions	user_id, date DESC	B-Tree
idx_trans_status	transactions	user_id, status	Partial (status='pending')
idx_splits_user	splits	user_id, is_settled	B-Tree
idx_budgets_user_cat	budgets	user_id, cat_id	B-Tree

5. SECURITY & PRIVACY

5.1 Encryption Layers

Layer	Method	Detail
SQLite DB file	AES-256 (SQLCipher)	Key derived from device Keystore + PIN via PBKDF2 (100,000 iterations, SHA
SecureStore	iOS Keychain / Android Keystore	System-managed hardware-backed secure storage for JWT tokens
Supabase Cloud	AES-256 (Supabase managed)	All stored data encrypted at rest; PII fields use column-level encryption
Transit	HTTPS / TLS 1.3	All API calls; certificate pinning via react-native-ssl-pinning
Backups	AES-256 + user passphrase	Optional encrypted export; passphrase never stored by the app

5.2 Biometric Authentication

- Implemented via **Expo LocalAuthentication** — iOS Face ID / Touch ID and Android BiometricPrompt.
- Enabled optionally in Settings → Security → Use Biometrics.
- Triggers on every cold app launch or after a 5-minute background timeout.
- Fallback to device PIN/pattern allowed after 3 consecutive biometric failures.
- Biometric preference stored as a boolean in Expo SecureStore (never in plain AsyncStorage).

5.3 GDPR Compliance

Right	Implementation
Right of Access	Settings → Export Data: generates a full JSON export of all user data
Right to Erasure	Settings → Delete Account: Edge Function hard-deletes all rows and the auth user
Data Minimisation	Only email + display name collected at signup. No location, no ad identifiers.
Consent	Explicit checkbox at signup for T&C and Privacy Policy; timestamp logged to DB
Data Residency	Supabase project hosted in ap-south-1 (Mumbai) for Indian user data compliance
Breach Notification	Supabase audit logs monitored; automated alert to admin email within 24h of anomaly

5.4 Privacy Logging Rules

- Transaction amounts and notes are **never** logged to Sentry or any analytics tool.
- User email is SHA-256 hashed before being sent to any third-party analytics service.
- No advertising SDK integrated; no IDFA/AAID collection on any platform.

6. TECHNOLOGY STACK

Layer	Technology	Version	Purpose
App Framework	React Native	0.74.x	Cross-platform iOS & Android
Dev Toolchain	Expo SDK	51.x	Build, OTA updates, native modules
Language	TypeScript	5.4.x	Static typing across entire codebase
Navigation	Expo Router (file-based)	3.x	Stack, Tab, and Modal navigation
State — Local	Zustand	4.x	In-memory UI state, user session
State — Server	TanStack React Query	5.x	Data fetching, caching, background sync
Local Database	Expo SQLite (SQLCipher)	13.x	Encrypted offline storage
Cloud Database	Supabase (PostgreSQL 15)	Latest	Cloud sync, Auth, Realtime, Storage
Auth	Supabase Auth	Latest	Email/Password + Google OAuth
Charts	Victory Native XL	40.x	Dashboard spending graph, analytics charts
Forms	React Hook Form + Zod	7.x/3.x	Form state & schema validation
Icons	@expo/vector-icons (Feather)	14.x	Consistent icon language
Push Notifications	Expo Notifications + FCM/APNs	Latest	Budget alerts, settlement reminders
Biometrics	Expo LocalAuthentication	14.x	Face ID / Touch ID / Fingerprint
Secure Storage	Expo SecureStore	13.x	JWT storage in hardware keystore
Background Tasks	Expo Background Fetch	12.x	Sync engine background execution
OTA Updates	Expo Updates (EAS Update)	0.25.x	Over-the-air JS bundle updates
CI/CD	GitHub Actions + EAS Build	Latest	Automated test, build, deploy pipeline
Crash Reporting	Sentry React Native	5.x	Error monitoring with PII scrubbing
Testing	Jest + React Native Testing Library	29.x	Unit and integration tests
E2E Testing	Maestro	1.x	End-to-end UI flows on simulator/device

7. API ENDPOINTS & INTEGRATION REFERENCE

All client-to-cloud communication uses the **Supabase REST API** (auto-generated by PostgREST) and **Supabase Edge Functions** (Deno runtime) for complex business logic. Base URL: https://[PROJECT_REF].supabase.co

7.1 Authentication Endpoints

Endpoint	Method	Description	Request Body
/auth/v1/token?grant_type=password	POST	Email/Password login	{ email, password }
/auth/v1/token?grant_type=refresh_token	POST	Silent token refresh	{ refresh_token }
/auth/v1/signup	POST	New user registration	{ email, password, data: { display_name } }
/auth/v1/recover	POST	Password reset email	{ email }
/auth/v1/logout	POST	Invalidate refresh token	Bearer token in Authorization header

7.2 Transactions REST Endpoints

Endpoint	Method	Query / Headers	Notes
/rest/v1/transactions	GET	?user_id=eq.{uid}&order=date.desc&limit=50	Paginated; cursor on 'date'
/rest/v1/transactions	POST	Content-Type: application/json	Batch insert via JSON array body
/rest/v1/transactions	PATCH	?id=eq.{id}	Partial update; send only changed fields
/rest/v1/transactions	POST	Prefer: resolution=merge-duplicates	Upsert on sync; conflict by updated_at

7.3 Supabase Edge Functions

Function	Trigger	Description
delete-account	POST (authenticated)	Hard-deletes all user data; deletes auth user; sends farewell email
send-settlement-reminder	Cron (every 24h)	Queries unsettled splits older than 48h; sends push notification to debtor
export-user-data	POST (authenticated)	Generates GDPR JSON export; returns signed Supabase Storage URL
budget-alert-check	DB trigger on INSERT	Compares new transaction against budget; fires push if threshold crossed
group-invite-email	POST (authenticated)	Sends email invite to non-registered users to join a group

7.4 Error Handling Matrix

HTTP Code	Scenario	Client Behaviour
400	Validation error	Show inline field error from response.message
401	Expired / invalid JWT	Attempt silent refresh; if fails → redirect to Login screen
403	RLS policy violation	Show generic 'Access Denied' toast; log to Sentry
409	Upsert conflict	Fetch server version; apply last-write-wins; re-sync
429	Rate limit exceeded	Exponential back-off; show 'Sync delayed' status banner
500	Supabase internal error	Retry 3× with back-off; show 'Cloud unavailable' banner; continue offline
Network	No connectivity	All writes to SQLite; show offline indicator chip in header

8. TESTING PROTOCOLS, CI/CD & DEPLOYMENT

All code must meet a minimum **80% line coverage** threshold enforced in CI. Critical paths (auth, sync engine, debt minimisation) require **95% coverage**. Tests run automatically on every pull request before merge is permitted.

8.1 Test Case Matrix

Module	Test ID	Scenario	Expected Result	Priority
Auth	AUTH-01	Valid login	JWT stored; Dashboard shown	P0
Auth	AUTH-02	Invalid password	Error toast; no token stored	P0
Auth	AUTH-03	Token expiry + silent refresh	New token; no re-login prompt	P0
Auth	AUTH-04	Biometric success	App unlocked immediately	P1
Transactions	TXN-01	Add expense while online	SQLite write + status=synced	P0
Transactions	TXN-02	Add expense while offline	SQLite write; status=pending	P0
Transactions	TXN-03	Sync on network reconnect	Pending rows pushed; synced	P0
Dashboard	DASH-01	Balance = Income - Expenses	Correct total displayed	P0
Group Split	GRP-01	Equal split among 3 members	Each owes exactly Total/3	P0
Group Split	GRP-02	Percentage split — sum not 100%	Validation error shown	P0
Group Split	GRP-03	Debt minimisation — 4 member group	No more than 3 settlements	P0
Budget	BUD-01	Spend reaches alert_pct threshold	Push notification fired	P1
Sync	SYNC-01	Conflict: same row edited on 2 devices	Higher updated_at wins	P0
Security	SEC-01	SQL injection attempt in note field	Parameterised query; no exec	P0
Security	SEC-02	Access another user's data via API	RLS returns 0 rows	P0

8.2 CI/CD Pipeline

Step	Action	Tool	On Failure
1	Lint + Format	ESLint + Prettier	Block PR merge
2	Type Check	tsc --noEmit	Block PR merge
3	Unit Tests	Jest	Block if coverage < 80%
4	E2E Tests	Maestro on EAS	Block production release
5	Preview Build	EAS Build	Notify Slack channel
6	Production Release	EAS Build + OTA	Manual approval gate required

8.3 Phased Deployment Plan

Phase	Milestone	Features	Target
Alpha	Internal dev	Auth, Add/View transactions, Dashboard, SQLite	Week 4
Beta	QA preview	Full CRUD, Spending graph, Categories, Offline sync	Week 8
v1.0	App Store launch	All Phase 1 features, Biometrics, Budget, Export	Week 12

Phase	Milestone	Features	Target
v1.1	Group Splits	Full group splitting module + debt minimisation	Week 16
v1.2	Analytics	Advanced charts, Hindi locale, PDF reports	Week 20